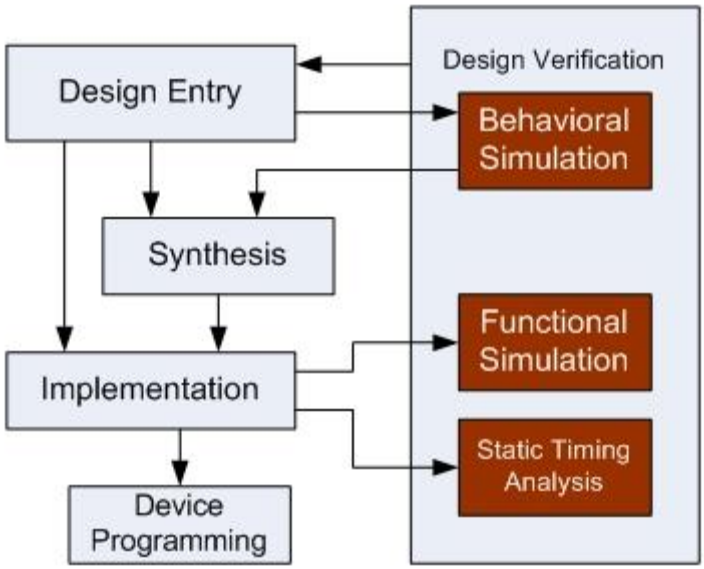
Module-2

❖ Explain the FPGA Design Flow

The design flow illustrates the pipeline for implementing and programming any given logic on the physical board. I chose to dub this design flow as FPGA Development Life Cycle or FDLC because of its analogy with SDLC.

FPGA architectural design flow comprises design entry, logic synthesis, design implementation, device programming, and design verification. However, the exact steps vary with manufactures.



Design Entry

The description of the logic can be made using either a schematic editor, a finite state machine (FSM) editor, or a hardware description language (HDL). This is done by selecting components from a given library and providing a direct mapping of the design functions to selected computing blocks.

When designs with a very large amount of function become difficult to manage graphically, HDL may be used to capture the design either in a structural or in a behavioral way. Besides VHDL and Verilog, which are the most established HDLs, several C-like languages are also available such as Handel-C, Impulse C, and SystemC.

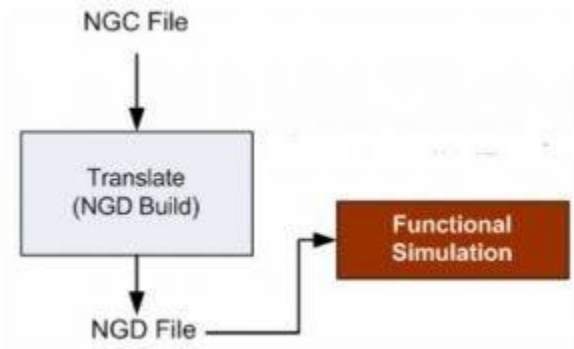
Logic Synthesis

This process translates the above VHDL code into a device netlist format for depicting a complete circuit with logical elements. Synthesis involves checking the code syntax and analyzing the hierarchy of the design architecture. Next, the code is compiled along with optimization and the generated netlist is saved as a *.ngc* file.

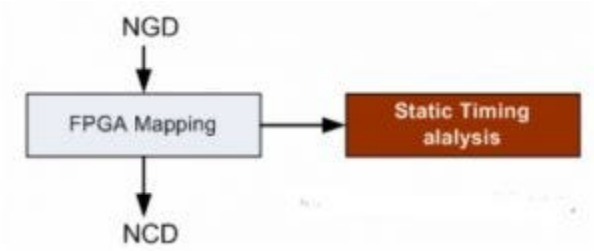
Design Implementation

The design implementation process consists of the following steps:

Translate*:* This process combines all the input netlists into the logic design file which is saved as a . ngd file. Here user constraints file assigns the ports to the physical elements.



- **Map***: This involves mapping the logic defined by the .ngd file into the components of FPGA and then generating a* . ncd *file.*



- ***Place and Route*:** Here routing places the sub-blocks from the above process into the logic blocks according to the constraints and then connect those blocks.

Device Programming

The above mentioned routed design must be loaded and converted into a format supported by the FPGA. Hence, the routed ***.ncd*** file is given to the BitGen program, which generates a bitstream file that contains all the programming information for an FPGA.

Design Verification

This is done all along with the design flow for ensuring that the logic behaves as intended. The following simulations are involved in this process:

- *Behavioral Simulation (RTL Simulation)*
- *Functional Simulation*
- *Static Timing Simulation*

These simulations are done in order to emulate the behavior of the components by providing test patterns to the inputs of the design and observing the outputs.

❖ **Explain the programming Technologies**

The Programming elements are used to implement the programmable connections among FPGA's logic blocks and a typical FPGA may contain more than 100,000 programming elements. For these reasons the elements should have the following properties:

- the programing element should consume as little chip area as possible,
- the programming element should have a low ON resistance and a very high OFF resistance,
- the programming element should contribute low parasitic capacitance to the wiring resources to which it is attached, and
- it should be possible to reliably fabricate a large number of programming elements on a single chip.

2.1.1 Static RAM Programming Technology

The static RAM programming technology is used in FPGAs produced by several companies: Algotronix, Concurrent Logic, Plessey Semiconductors, and Xilinx. In these FPGAs, programmable connections are made using pass-transistors, transmission gates, or multiplexers that are all controlled by SRAM cells. The use of a static RAM cell to control a CMOS pass-transistor is illustrated in Figure 2.2a. Alternatively, the RAM cell could control both the n-channel and p-channel transistors of a full transmission gate, or more than one RAM cell could be used to control the select inputs on a multiplexer. The later two options are illustrated by Figures 2.2b and 2.2c.

In the case of the pass-transistor approach in Figures 2.2a and 2.2b, the RAM cell controls whether the pass-gates are on or off. When off, the pass-gate presents a very high resistance between the two wires to which it is attached. When the pass gate is turned on, it forms a relatively low resistance connection between the two wires. For the multiplexer approach in Figure 2.2c, the RAM cells control which of the multiplexer's inputs should be connected to its output. This scheme would typically be used to optionally connect one of several wires to a single input of a logic block.

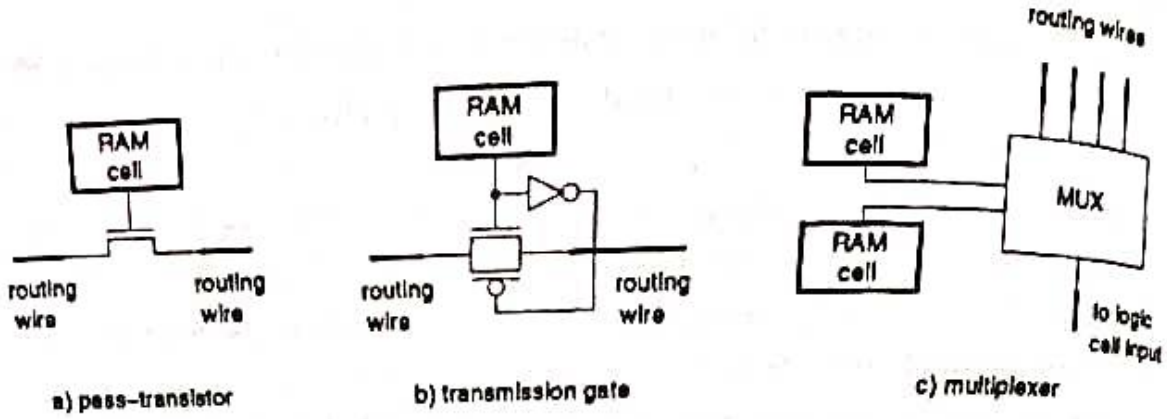


Figure 2.2 - *Static RAM Programming Technology.*

In an FPGA that uses the SRAM programming technology, the logic blocks may be interconnected using a combination of pass-gates and multiplexers. Since the static RAM is volatile, these FPGAs must be configured each time the power is applied to the chip. This implies that a system that includes such chips must have some sort of permanent storage mechanism for the RAM cell bits, such as a ROM or a disk. The RAM cells bits may be loaded into the FPGA either through a serial arrangement (if the RAM cells can be arranged in series during chip configuration) or each RAM cell may be addressed as an element of an array (as in a normal static RAM chip).

Compared with other programming technologies described in this section, the chip area required by the static RAM approach is relatively large. This is because at least five transistors are needed for each RAM cell, as well as the additional transistors for the pass-gates or multiplexers. The major advantage of this technology is that it provides an FPGA that can be reconfigured (in-circuit) very quickly and it can be produced using a standard CMOS process technology.

2.1.2 Anti-fuse Programming Technology

Anti-fuse programming technology is used in FPGAs offered by Actel Corp., QuickLogic, and Crosspoint Solutions. While the anti-fuse used in each of these FPGAs differs in construction, their function is the same. An anti-fuse normally resides in a high-impedance state but can be "fused" into a low-impedance state when programmed by a high voltage. In the remainder of this sub-section, the construction and use of the anti-fuses used by Actel and Quicklogic are described.

The Actel anti-fuse, called PLICE, is described in detail in [Hamd88]. It can be described as a square structure that consists of three layers: the bottom layer is composed of positively-doped silicon (n+ diffusion), the middle layer is a dielectric (Oxygen-Nitrogen-Oxygen insulator), and the top layer is made of poly silicon. This construction is illustrated by Figure 2.3a.

The PLICE anti-fuse is programmed by placing a relatively high voltage (18 V) across the anti-fuse terminals and driving a current of about 5 ma through the device. This procedure generates enough heat in the dielectric to cause it to melt and form a conductive link between the Poly-Si and n+ diffusion. Special high-voltage transistors are fabricated within the FPGA to accommodate the necessary large voltages and currents.

Both the bottom layer and top layer of the anti-fuse are connected to metal wires, so that, when programmed, the anti-fuse forms a low resistance connection (from 300 to 500 ohms) between the two metal wires. This arrangement is depicted in Figure 2.3b. The PLICE anti-fuse is manufactured by adding three specialized masks to a normal CMOS process.

The anti-fuse used by Quicklogic is called ViaLink. As described in [Birk91], it is similar to the PLICE anti-fuse in that it consists of three layers. However, a ViaLink anti-fuse uses one level of metal for its bottom layer, an alloy of amorphous silicon for its middle layer, and a second level of metal for the top layer. This structure is illustrated in Figure 2.4. When in the unprogrammed state, the anti-fuse presents over a gigaohm of resistance, but when programmed it forms a low-resistance path of about 80 ohms between the two metal wires. The anti-fuse is manufactured using three extra masks

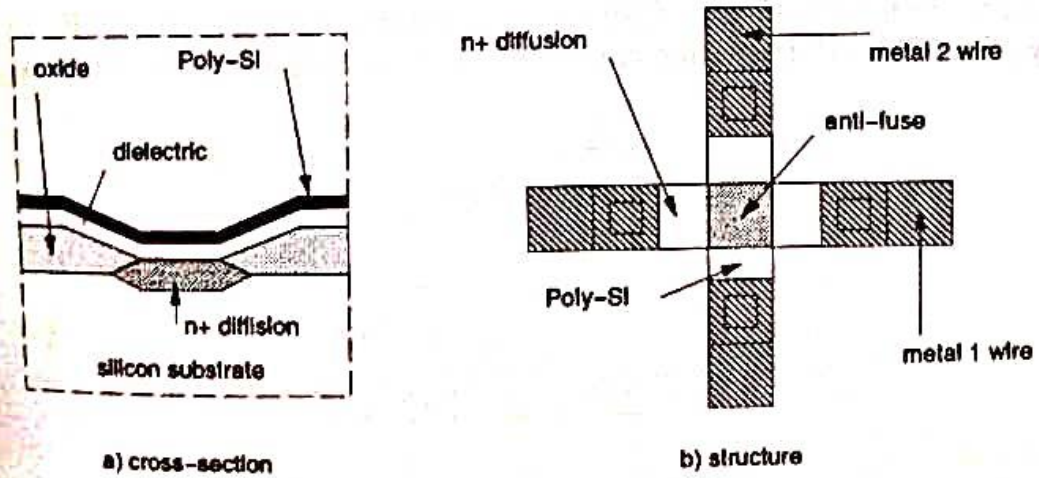


Figure 2.3 - *PLICE Anti-fuse Programming Technology.*

above a normal CMOS process. Here, a normal via is created for the anti-fuse, but the via is filled with the amorphous silicon alloy instead of metal.

The ViaLink anti-fuse is programmed by placing about 10 volts across its terminals. When sufficient current is supplied, this results in a change of state in the amorphous silicon and creates a conductive link between the bottom and top layers of metal.

The chip area required by an anti-fuse (either PLICE or ViaLink) is very small compared to the other programming technologies. However, this is somewhat offset by the large space required for the high-voltage transistors that are needed to handle the high programming voltages and currents. A disadvantage of anti-fuses is that their manufacture requires modifications to the basic CMOS process. Their properties are summarized in the table at the end of this section.

2.1.3 EPROM and EEPROM Programming Technology

EPROM programming technology is used in the FPGAs manufactured by Altera Corp. and Plus Logic. This technology is the same as that used in EPROM memories. Unlike a simple MOS transistor, an EPROM transistor comprises two gates, a *floating* gate and a *select* gate. The floating gate, positioned between the select gate and the transistor's channel, is so named because it is not electrically connected to any circuitry. In its normal (unprogrammed) state, no charge exists on the floating gate and the transistor can be turned ON in the normal fashion using the select gate. However, when the transistor is programmed by causing a large current to flow between the source and drain, a charge is trapped under the floating gate. This charge has the effect of permanently turning the transistor OFF. In this way, the EPROM transistor can function as a programmable element. An

EPROM transistor can be re-programmed by first removing the trapped charge from the floating gate. Exposing the gate to ultraviolet light excites the trapped electrons to the point where they can pass through the gate oxide into the substrate.

EPROM transistors are used in FPGAs in a different manner than are static RAM cells or anti-fuses. That is, rather than serving to programmably connect two wires, EPROM transistors are used as "pull down" devices for logic block inputs. This arrangement is illustrated in Figure 2.5. As the figure shows, one wire, called the "word line" (using memory terminology), is connected to the select gate of the EPROM transistor. As long as the transistor has not been programmed into the OFF state, the word line can cause the "bit line", which is connected to a logic block input to be pulled to logic zero. In both the Altera and Plus Logic FPGAs, many EPROM transistors, each driven by a different word line, are connected to the same bit line. Since a pull-up resistor is present on the bit line, this scheme allows the EPROM transistors to not only implement connections but also to realize wired-AND logic functions. A disadvantage of this approach is that the resistor consumes static power.

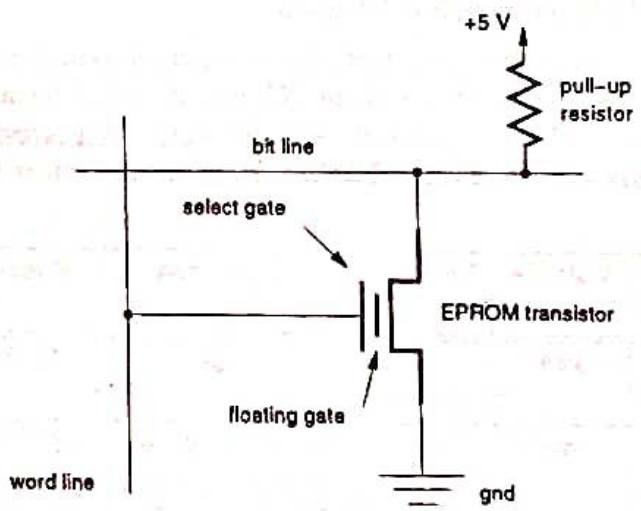


Figure 2.5 - *EPROM Programming Technology.*

One advantage of EPROM transistors is that they are re-programmable but do not require external storage. However, unlike static RAM, EPROM transistors cannot be re-programmed in-circuit.

The EEPROM approach (which is used in the FPGAs offered by Advanced Micro Devices) is similar to the EPROM technology except that EEPROM transistors can be re-programmed in-circuit. The disadvantage of using EEPROM transistors is that they consume about twice the chip area as

EPROM transistors and they require multiple voltage sources (for reprogramming) which might not otherwise be required.

❖ Explain the configurable logic block

(Ans) The array of CLBs provides the functional elements from which the user's logic

is constructed. The logic blocks are arranged in a matrix within the perimeter of IOBs. [For example, the XC3020A has 64 such blocks arranged in 8 rows and 8 columns.] The development system is used to compile the configuration data which is to be loaded into the internal configuration memory to define the operation and interconnection of each block. User definition of CLBs and their interconnecting networks may be done by automatic translation from a schematic-capture logic diagram or optionally by installing library or user macros.

Each CLB has a combinatorial logic section, two flip-flops, and an internal control section. See Fig. 4.4. There are: five logic inputs (A, B, C, D and E); a common clock input (K); an asynchronous direct RESET input (RD); and an enable clock (EC). All may be driven from the interconnect resources adjacent to the blocks. Each CLB also has two outputs (X and Y) which may drive interconnect networks. Data input for either flip-flop within a CLB is supplied from the function F or G outputs of the combinatorial logic, or the block input, DI. Both flip-flops in each CLB share the asynchronous RD which, when enabled and High, is dominant over clocked inputs. All flip-flops are reset by

the active-Low chip input, RESET, or during the configuration process. The flip-flops share the enable clock (EC) which, when Low, recirculates the flip flops' present states and inhibits response to the data-in or combinatorial function inputs on a CLB. The user may enable these control inputs and select their sources. The user may also select the clock net input (K), as well as its active sense within each CLB. This programmable inversion eliminates the need to route both phases of a clock signal throughout the device.

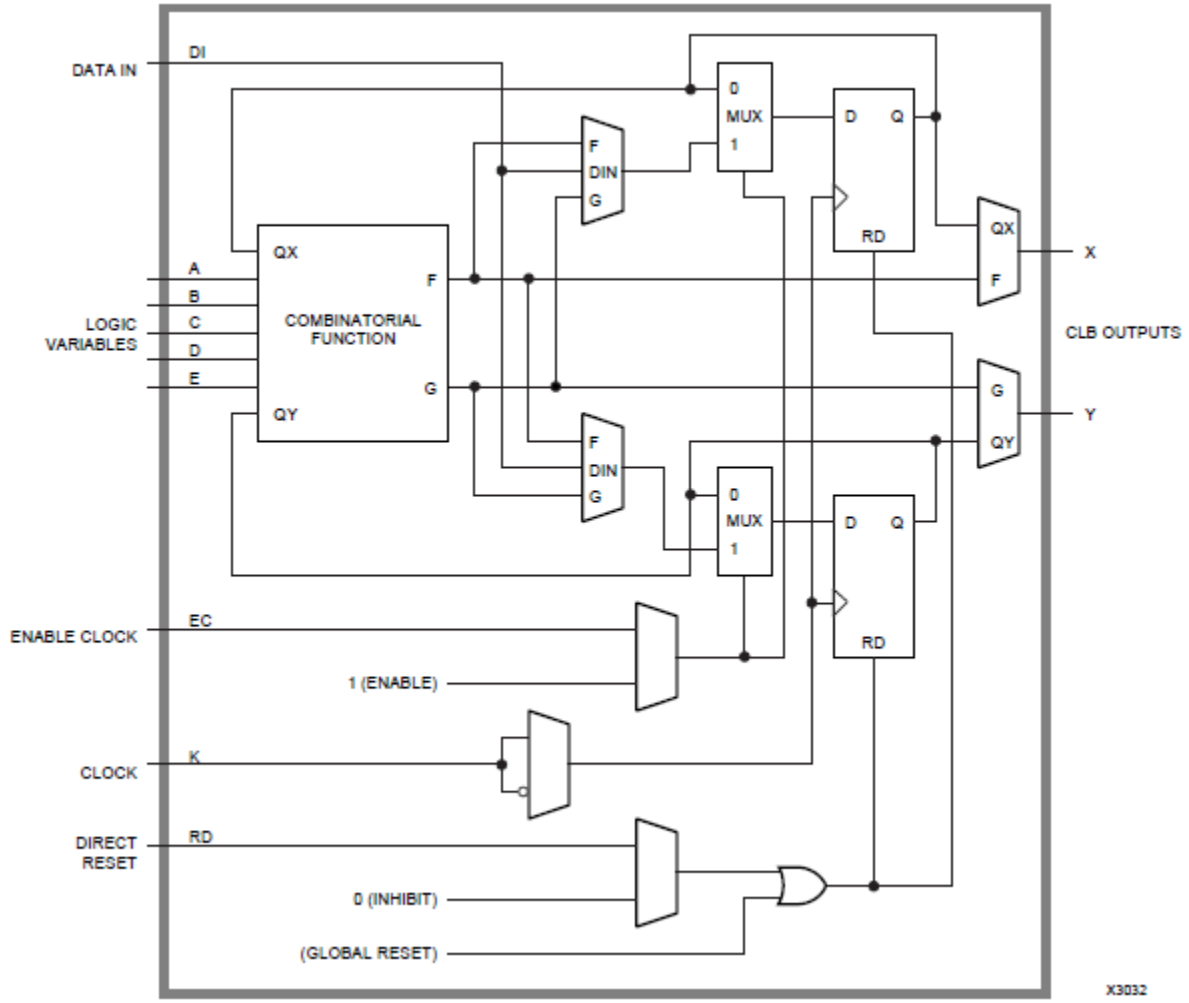


Fig. 4.4 Xilinx: XC3000 series CLB

❖ Explain programmable interconnects & Configurable IO blocks

Programmable Interconnects:

Implementing an entire design on the LCA requires interconnecting the various CLBs and IOBs. This facilitated by the programmable interconnect resources, which consist of a grid of two layers of metal segments, programmable interconnect points (PIPs) and switch boxes. A PIP comprises in Fig. 4.7. Dropping a "1" into the RAM Cell establishes a connection between two points.

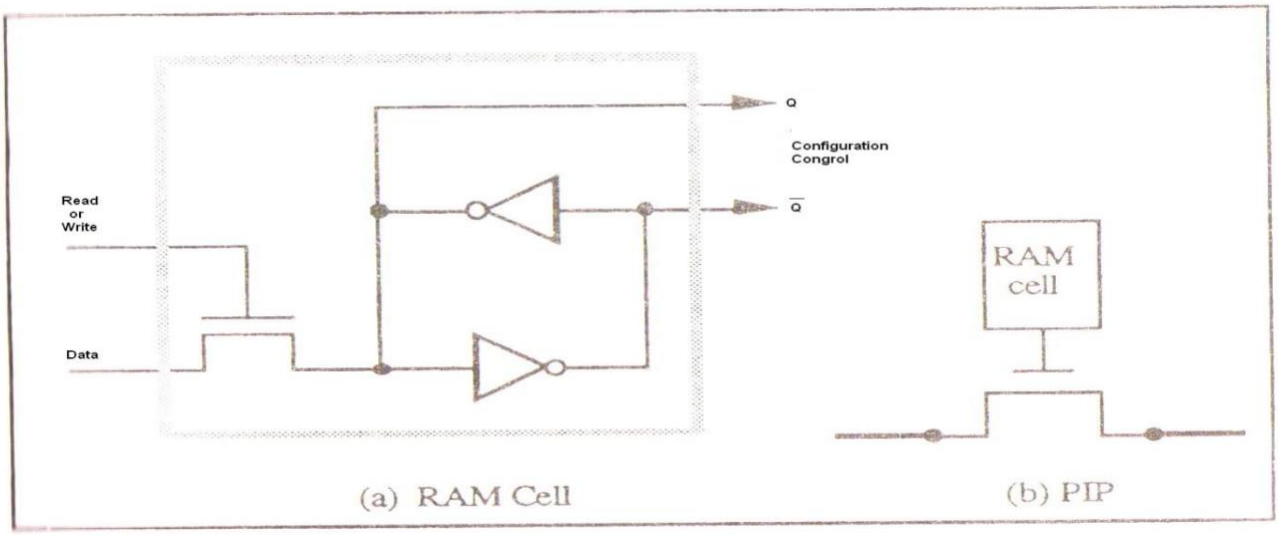


Fig. 4.7 The Programmable Interconnect: RAM Cell and PIP

Different routing resources are available for different circumstances. Vertical and horizontal long lines run the entire height and width of the interconnect area. They used to carry high-fanout signals or signals that need to travel long distance with low skew. Direct connection is available between adjacent CLBs and IOBs for short-distance and low-delay communication. General-purpose interconnect is the most versatile of all. It consists of a network of switching matrices and wore segments to facilitate general network branching and routing.

The general-purpose interconnects are to the Xilinx device what the routing channels are to gate arrays. In the XC3000 family, they consist of five horizontal and five vertical lines located between the rows and columns of CLBs. At the intersection of vertical and horizontal lines, there is a switch box, also called a switching matrix, as shown in the Fig. 4.8. Twenty admissible interconnect options of a Xilinx XC3000 switch box are depicted in Fig. 4.9. Some signal-restoring buffers are available along some routing resources to restore signal levels due to degradation through the pass transistors inside the switch boxes.

Each XC2000 and XC3000 CLB can be directly connected to its neighboring blocks. The X-output of a CLB is directly connected to the B-output of the CLB immediately to its right and the C-input of the CLB to its left. The Y-output is connected to the A- and D- input of the CLB below and above, respectively.

Direct interconnect is also provided between the CLBs (along the logic cell boundary) and some IOBs. Direct interconnect typically has less routing delay than general-purpose interconnect and long lines.
Long interconnect lines or long lines are available along the vertical and horizontal channels. They bypass the switch boxes and have less skew than general-purpose interconnect. The horizontal long lines in a XC3000 or XC4000 device may be connected to the tri-state buffers that are accessible to CLBs. This feature allows the formation of busses, multiplexers, and wired AND functions. As in the case of direct interconnections, long lines have the advantage of minimizing delays for high-fanout nets and lowering skew. The interconnect facility in the series 4000 devices is significantly different. In particular, the switch box connections are much simpler.

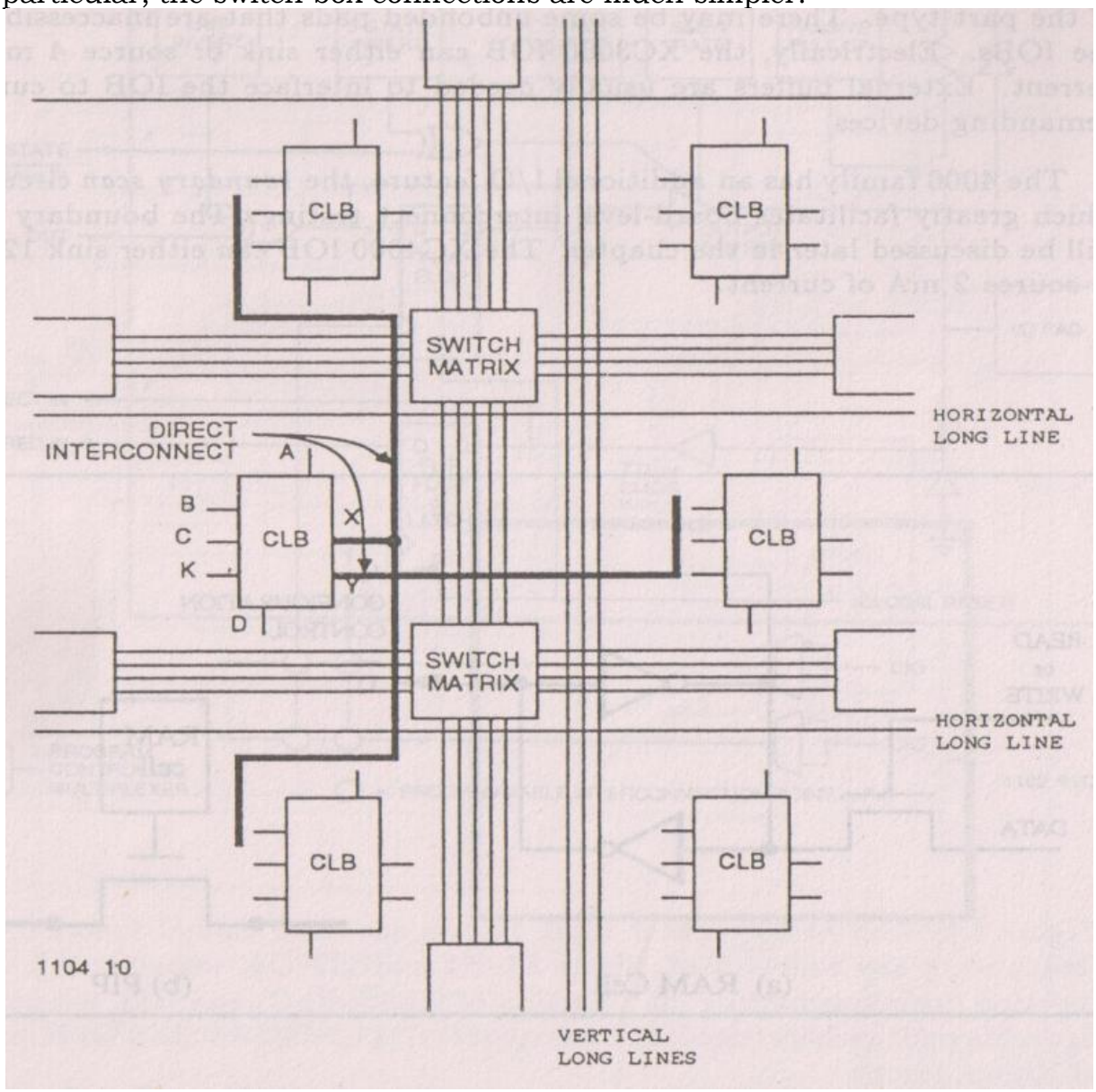


Fig. 4.8 Xilinx: General Interconnects

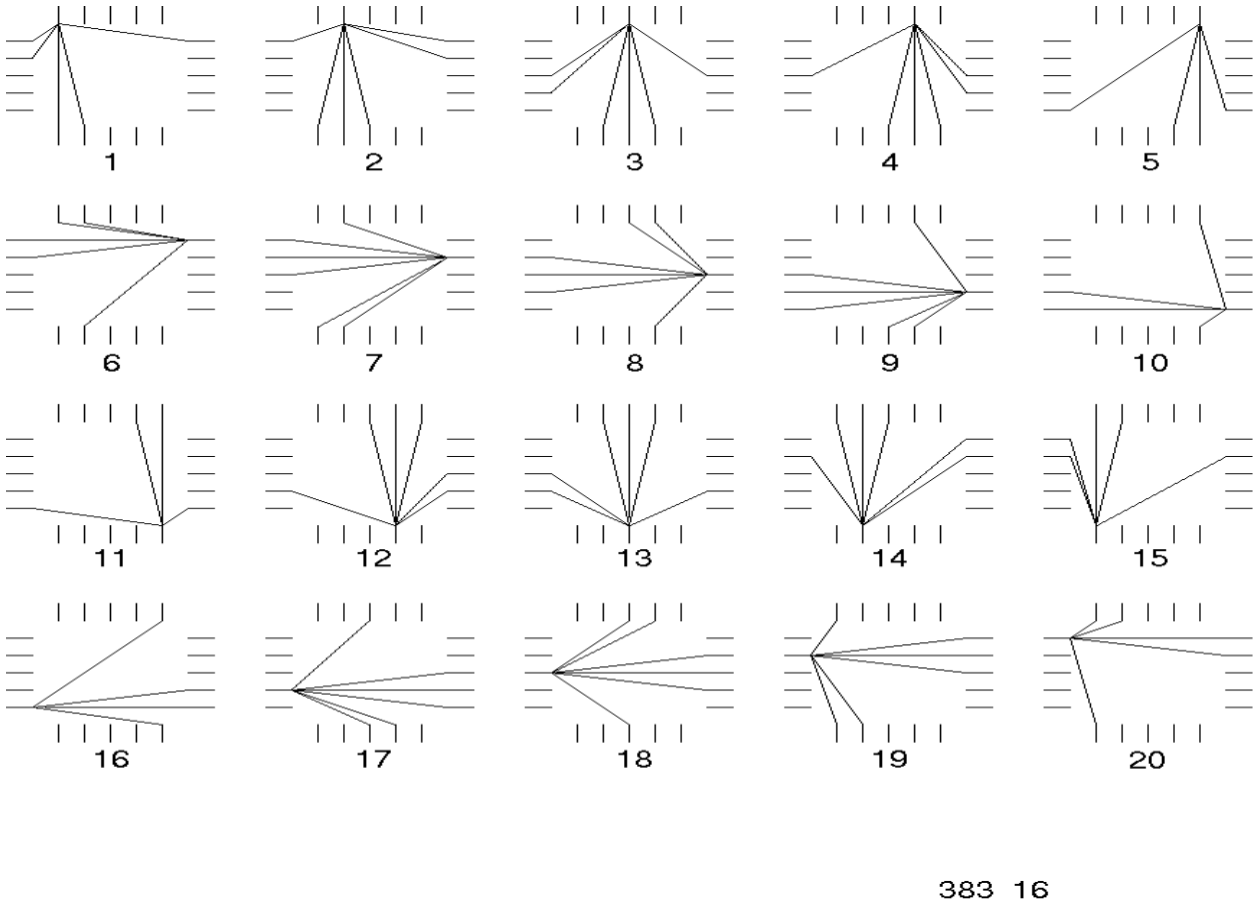


Fig. 4.9 Possible switch box connectivities

Configurable IO blocks

The XC3000 IOB, is shown in Fig. 4.6, provides an interface between the Logic Cell Array (LCA) and the external world. There is one IOB for every programmable pin on the LCA package. The XC3090, for instance, provides 144 such I/O pins. Each IOB contains input and output buffers to facilitate compatibility with TTL and CMOS threshold levels. The IOB can serve as an input, output or tri-stated bidirectional path. The input signal can be introduced to the internal logic directly or via a D-flip-flop (registered) or a transparent latch. Similarly, the output can optionally be registered or directly connected to the pad. The output buffer is provided with skew control as well as tri-state control. Generally the number of I/O pads and IOBs increase with the logic capacity of the part type. There may be some unbounded pads that are inaccessible to the IOBs. Electrically, the XC3000 IOB can either sink or source 4mA of current. External buffers are usually needed to interface the IOB to current demanding devices.
The 4000 family has an additional I/O feature, the boundary scan circuitry, which greatly facilitates board-level interconnect testing. The XC4000 IOB can

either sink 12 mA or source 2 mA of current.

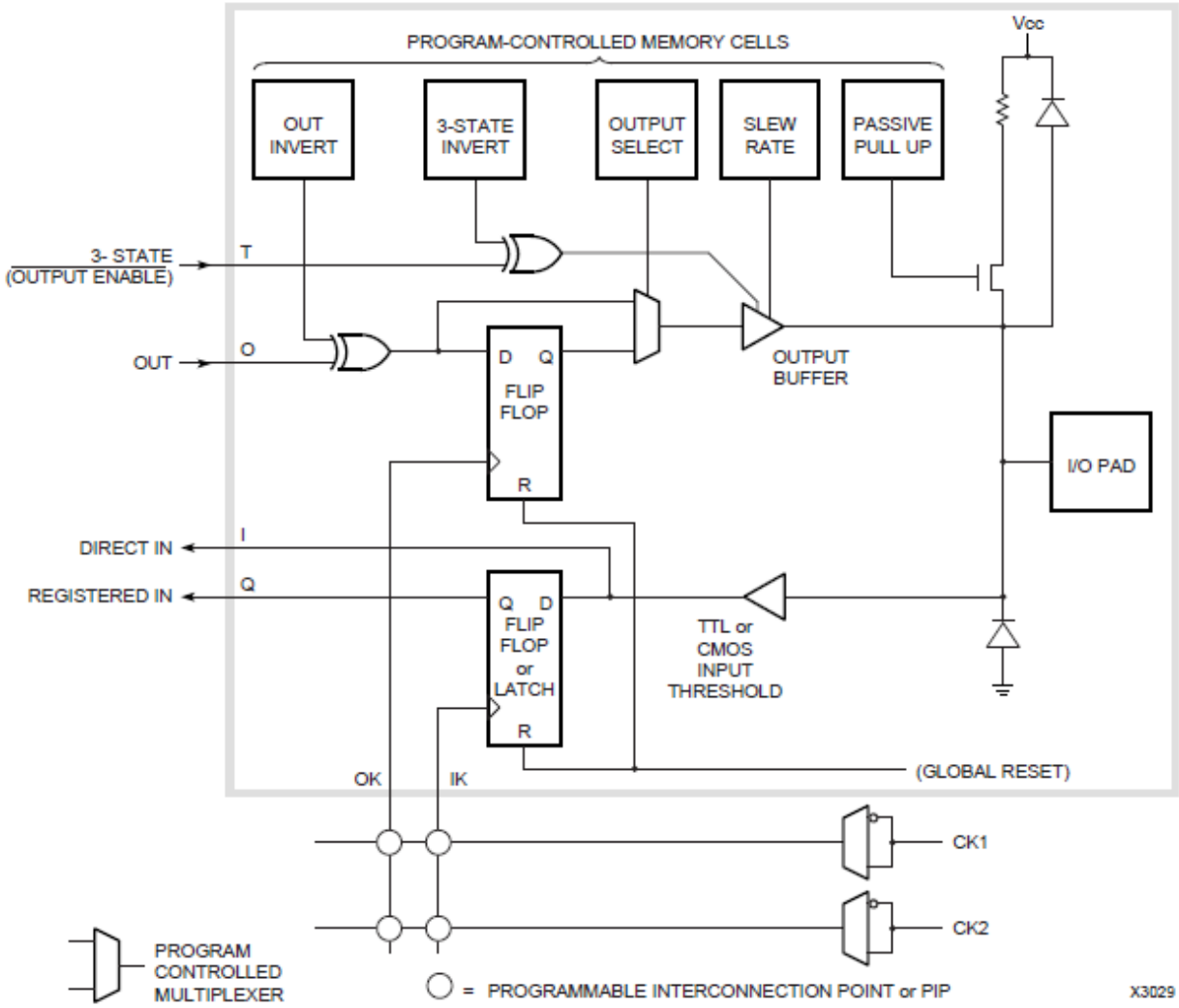


Fig. 4.6 The I/O block of Xilinx: XC3000 series CLB

Discuss logic synthesis with Boolean diagram

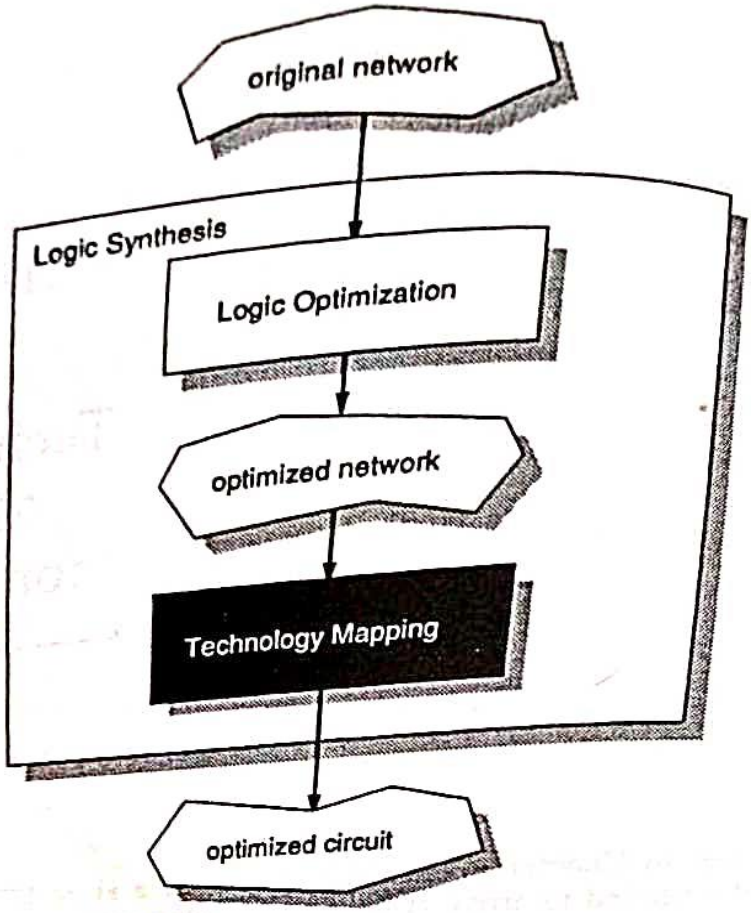


Figure 3.1 - *Logic Synthesis.*

3.1 Logic Synthesis

In this chapter, it is assumed that the original network to be manipulated by the logic synthesis tools is composed of a number of combinational functions. A network that contains flip-flops can also be processed, but it would first be broken up into a set of combinational functions at flip-flop boundaries. When read by the logic optimization phase, the network specifies a set of primary inputs that feed the combinational functions, and a set of primary outputs that are generated by the combinational functions.

One way to represent a set of combinational functions is to use a Directed Acyclic Graph (DAG) referred to as a Boolean network. An example of a small Boolean network is given in Figure 3.2. This network specifies a combinational function that has five primary inputs, *a*, *b*, *c*, *d*, *e*, and one

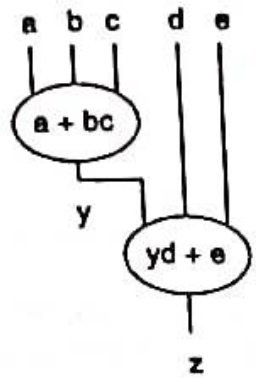


Figure 3.2 - *A Boolean Network.*

primary output, z. Each node in the Boolean network defines a function represented by a variable associated with the node. This function can be specified as a *local function* of the node's inputs, or as a *global function* of the network's primary inputs. For example, the node y in Figure 3.2 represents the local function $y = a + bc$, and the node z represents the global function $z = (a + bc)d + e$.

As Figure 3.1 illustrates, the end result of logic synthesis is an optimized circuit implementing the original Boolean network. The optimization goal is typically based on a measure of the area of the circuit, its speed performance, or both. The final circuit is also represented by a Boolean network. In the FPGA environment, each node represents an FPGA logic block and the local function of the node specifies the function implemented by the logic block.

3.1.1 Logic Optimization

The initial phase of many logic synthesis systems, for example misII [Bray87] and BOLD [Bost87], restructures the original network to reduce a cost function that is calculated directly from the network itself. The intention is to improve the final circuit by reducing the complexity of the network. Since this phase does not consider the type of element that will be used for the final circuit, it is called *technology-independent* logic optimization. The modifications applied to the network typically include redundancy removal and common subexpression elimination. Logic optimization may also exploit don't cares in the specification of the desired combinational function to simplify the network.

In the misII synthesis system the complexity of a network is measured by counting the number of *literals* in the local function for each node. Each local function is a sum-of-products expression and each instance of a variable counts as one literal. For example, the following 4-input, 2-output network has 11 literals

$$f = ac + ad + bc + bd$$

$$g = a + b + c$$

The complexity of this network can be reduced by the following modifications. The expression $(a+b)$ is factored out of the equations for nodes f and g, and a new node e, implementing the function $a+b$, is created. The variable e is then substituted back into the equations for nodes f and g, resulting is the following 7 literal network.

$$e = a + b$$

$$f = e(c + d)$$

$$g = e + c$$

3.1.2 Technology Mapping

After logic optimization has produced the optimized network, technology mapping transforms this network into the final circuit. This is done by selecting pieces of the network that can each be implemented by one of the available circuit elements, and specifying how these elements are to be interconnected. The circuit is optimized to reduce a cost function that typically incorporates measures of both area and delay. Conventional approaches to technology mapping have focused on using circuit elements from a limited set of simple gates, such as a Standard Cell library. The complex logic blocks used in FPGAs present difficulties for library-based approaches because they can each implement a large number of functions. The next section briefly discusses library-based technology mapping, and the following sections describe technology mapping programs developed specifically for lookup table and multiplexer-based FPGAs.

Draw the mapping using Dynamic Programming

3.1.2 Technology Mapping

After logic optimization has produced the optimized network, technology mapping transforms this network into the final circuit. This is done by selecting pieces of the network that can each be implemented by one of the available circuit elements, and specifying how these elements are to be interconnected. The circuit is optimized to reduce a cost function that typically incorporates measures of both area and delay. Conventional approaches to technology mapping have focused on using circuit elements from a limited set of simple gates, such as a Standard Cell library. The complex logic blocks used in FPGAs present difficulties for library-based approaches because they can each implement a large number of functions. The next section briefly discusses library-based technology mapping, and the following sections describe technology mapping programs developed specifically for lookup table and multiplexer-based FPGAs.

3.1.2.1 Library-Based Technology Mapping

An important advance in technology mapping for conventional technologies was the formalization introduced by Keutzer in DAGON [Keut87], and also used in misII [Detj87]. In this formalization, the set of available

circuit elements is represented as a library of functions and the construction of the optimized circuit is divided into three subproblems: decomposition, matching and covering.

The original network is first decomposed into a canonical 2-input NAND representation. This decomposition guarantees that there will be no nodes in the network that are too large to be implemented by any library element, provided that the library includes a 2-input NAND. Note, however, that there are many possible 2-input NAND decompositions and that the one selected may not be the best decomposition.

After decomposition, the network is partitioned into a forest of trees, the optimal subcircuit covering each tree is constructed, and finally the circuit covering the entire network is assembled from these subcircuits. To form the forest of trees, the decomposed network is partitioned at fanout nodes into a set of single-output subnetworks. Each of these subnetworks is either a tree of a leaf-DAG. A leaf-DAG is a multi-input single-output DAG where only the input nodes have fanout greater than one. Each leaf-DAG is converted into a tree by creating a unique instance of every input node for each of its multiple fanout edges.

The optimal circuit implementing each tree is constructed using a dynamic programming traversal that proceeds from the leaf nodes to the root node. For every node in the tree an optimal circuit implementing the subtree extending from the node to the leaf nodes is constructed. This circuit consists of a library element that matches a subfunction rooted at the node and previously constructed circuits implementing its inputs. The cost of the circuit is calculated from the cost of the matched library element and the cost of the circuits implementing its inputs. To find the lowest-cost circuit, DAGON first finds all library elements that match subfunctions rooted at the node. The cost of the circuit using each of the candidate library elements is then calculated and the lowest cost circuit is retained. The set of candidate library elements is found by searching through the library and using tree matching [Aho85] to determine if each library element matches a subfunction rooted at the node.

As an example of the above procedure, consider the library shown in Figure 3.3 and the Boolean network in Figure 3.4. In this example, the library elements are standard cells and costs are given in terms of the area of the cells. The cost of the INV, NAND-2 and AOI-21 cells are 2, 3, and 4 respectively. In Figure 3.4a, the only library element matching at node E is the NAND-2 and the cost of the optimal circuit implementing node E is therefore 3. At node C the only matching library element is also the NAND-2. The cost of this NAND-2 is 3 and the cost of the optimal circuit

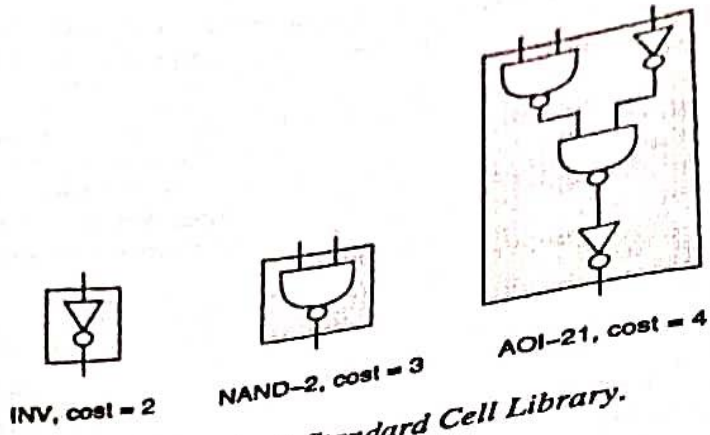


Figure 3.3 - *A Standard Cell Library.*

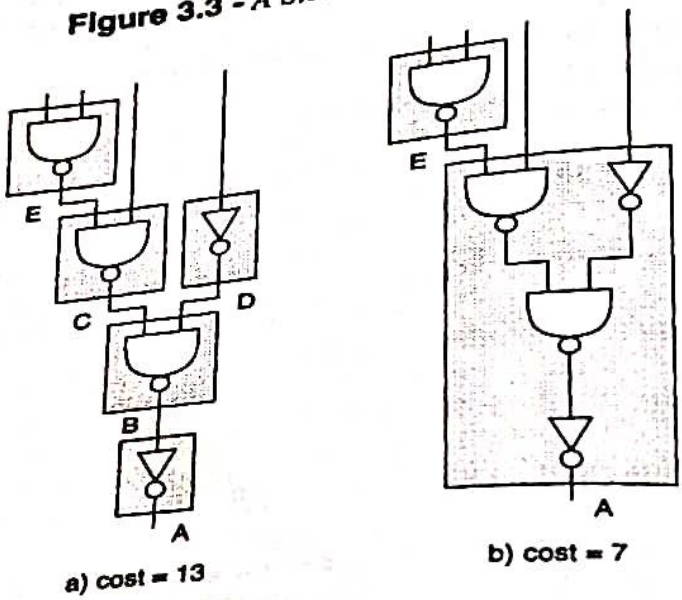


Figure 3.4 - *Mapping using Dynamic Programming.*

implementing its input E is also 3. Therefore, the cumulative cost of the optimal circuit implementing node C is 6.

Eventually, the algorithm will reach node A. For node A there are two matching library elements, the INV, as used in Figure 3.4a, and the AOI-21,

as used in Figure 3.4b. The circuit constructed using the INV matching A includes a NAND-2 implementing node B, a NAND-2 implementing node C, an INV implementing node D, and a NAND-2 implementing node E. The cumulative cost of this circuit is 13. The circuit constructed using the AOI-21 matching A includes a NAND-2 implementing node E. The cumulative cost of this circuit is 7. The circuit using the AOI-21 is therefore the optimal circuit for implementing node A.

The tree matching algorithm requires that each library function be represented as a 2-input NAND decomposition. For some functions, however, there are many possible decompositions. The inclusion of all decompositions can significantly increase the size of the library and the computational cost of the matching algorithm.

❖ **Discuss Static Ram Programming Technology**

2.1.1 Static RAM Programming Technology

The static RAM programming technology is used in FPGAs produced by several companies: Algotronix, Concurrent Logic, Plessey Semiconductors, and Xilinx. In these FPGAs, programmable connections are made using pass-transistors, transmission gates, or multiplexers that are all controlled by SRAM cells. The use of a static RAM cell to control a CMOS pass-transistor is illustrated in Figure 2.2a. Alternatively, the RAM cell could control both the n-channel and p-channel transistors of a full transmission gate, or more than one RAM cell could be used to control the select inputs on a multiplexer. The later two options are illustrated by Figures 2.2b and 2.2c.

In the case of the pass-transistor approach in Figures 2.2a and 2.2b, the RAM cell controls whether the pass-gates are on or off. When off, the pass-gate presents a very high resistance between the two wires to which it is attached. When the pass gate is turned on, it forms a relatively low resistance connection between the two wires. For the multiplexer approach in Figure 2.2c, the RAM cells control which of the multiplexer's inputs should be connected to its output. This scheme would typically be used to optionally connect one of several wires to a single input of a logic block.

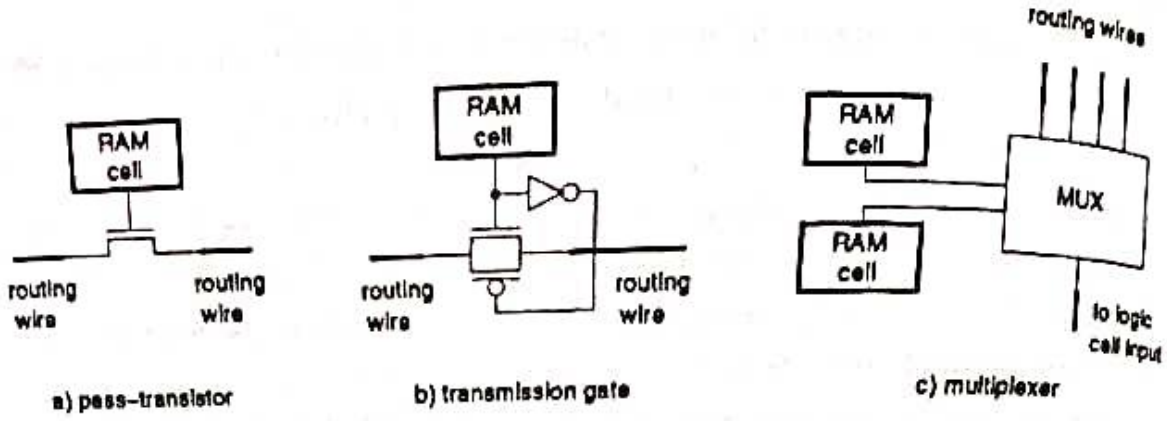


Figure 2.2 - *Static RAM Programming Technology.*

In an FPGA that uses the SRAM programming technology, the logic blocks may be interconnected using a combination of pass-gates and multiplexers. Since the static RAM is volatile, these FPGAs must be configured each time the power is applied to the chip. This implies that a system that includes such chips must have some sort of permanent storage mechanism for the RAM cell bits, such as a ROM or a disk. The RAM cells bits may be loaded into the FPGA either through a serial arrangement (if the RAM cells can be arranged in series during chip configuration) or each RAM cell may be addressed as an element of an array (as in a normal static RAM chip).

Compared with other programming technologies described in this section, the chip area required by the static RAM approach is relatively large. This is because at least five transistors are needed for each RAM cell, as well as the additional transistors for the pass-gates or multiplexers. The major advantage of this technology is that it provides an FPGA that can be reconfigured (in-circuit) very quickly and it can be produced using a standard CMOS process technology.

❖ **Discuss Anti –fuse programming Technology**

2.1.2 Anti-fuse Programming Technology

Anti-fuse programming technology is used in FPGAs offered by Actel Corp., QuickLogic, and Crosspoint Solutions. While the anti-fuse used in each of these FPGAs differs in construction, their function is the same. An anti-fuse normally resides in a high-impedance state but can be "fused" into a low-impedance state when programmed by a high voltage. In the remainder of this sub-section, the construction and use of the anti-fuses used by Actel and Quicklogic are described.

The Actel anti-fuse, called PLICE, is described in detail in [Hamd88]. It can be described as a square structure that consists of three layers: the bottom layer is composed of positively-doped silicon (n+ diffusion), the middle layer is a dielectric (Oxygen-Nitrogen-Oxygen insulator), and the top layer is made of poly silicon. This construction is illustrated by Figure 2.3a.

The PLICE anti-fuse is programmed by placing a relatively high voltage (18 V) across the anti-fuse terminals and driving a current of about 5 ma through the device. This procedure generates enough heat in the dielectric to cause it to melt and form a conductive link between the Poly-Si and n+ diffusion. Special high-voltage transistors are fabricated within the FPGA to accommodate the necessary large voltages and currents.

Both the bottom layer and top layer of the anti-fuse are connected to metal wires, so that, when programmed, the anti-fuse forms a low resistance connection (from 300 to 500 ohms) between the two metal wires. This arrangement is depicted in Figure 2.3b. The PLICE anti-fuse is manufactured by adding three specialized masks to a normal CMOS process.

The anti-fuse used by Quicklogic is called ViaLink. As described in [Birk91], it is similar to the PLICE anti-fuse in that it consists of three layers. However, a ViaLink anti-fuse uses one level of metal for its bottom layer, an alloy of amorphous silicon for its middle layer, and a second level of metal for the top layer. This structure is illustrated in Figure 2.4. When in the unprogrammed state, the anti-fuse presents over a gigaohm of resistance, but when programmed it forms a low-resistance path of about 80 ohms between the two metal wires. The anti-fuse is manufactured using three extra masks

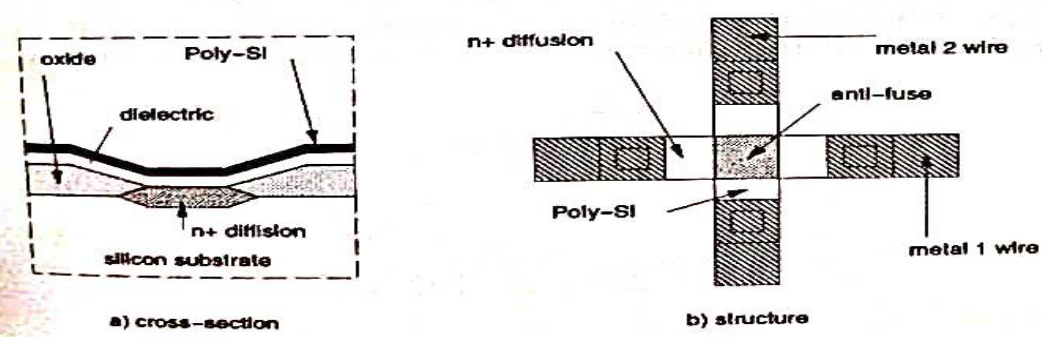


Figure 2.3 - *PLICE Anti-fuse Programming Technology.*

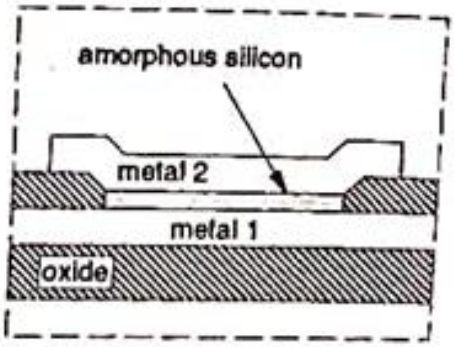


Figure 2.4 - *ViaLink Anti-fuse Programming Technology.*

above a normal CMOS process. Here, a normal via is created for the anti-fuse, but the via is filled with the amorphous silicon alloy instead of metal.

The ViaLink anti-fuse is programmed by placing about 10 volts across its terminals. When sufficient current is supplied, this results in a change of state in the amorphous silicon and creates a conductive link between the bottom and top layers of metal.

The chip area required by an anti-fuse (either PLICE or ViaLink) is very small compared to the other programming technologies. However, this is somewhat offset by the large space required for the high-voltage transistors that are needed to handle the high programming voltages and currents. A disadvantage of anti-fuses is that their manufacture requires modifications to the basic CMOS process. Their properties are summarized in the table at the end of this section.

❖ **Explain EPROM & EEPROM Programming Technology**

2.1.3 EPROM and EEPROM Programming Technology

EPROM programming technology is used in the FPGAs manufactured by Altera Corp. and Plus Logic. This technology is the same as that used in EPROM memories. Unlike a simple MOS transistor, an EPROM transistor comprises two gates, a *floating* gate and a *select* gate. The floating gate, positioned between the select gate and the transistor's channel, is so named because it is not electrically connected to any circuitry. In its normal (un-programmed) state, no charge exists on the floating gate and the transistor can be turned ON in the normal fashion using the select gate. However, when the transistor is programmed by causing a large current to flow between the source and drain, a charge is trapped under the floating gate. This charge has the effect of permanently turning the transistor OFF. In this way, the EPROM transistor can function as a programmable element. An

EPROM transistor can be re-programmed by first removing the trapped charge from the floating gate. Exposing the gate to ultraviolet light excites the trapped electrons to the point where they can pass through the gate oxide into the substrate.

EPROM transistors are used in FPGAs in a different manner than are static RAM cells or anti-fuses. That is, rather than serving to programmably connect two wires, EPROM transistors are used as "pull down" devices for logic block inputs. This arrangement is illustrated in Figure 2.5. As the figure shows, one wire, called the "word line" (using memory terminology), is connected to the select gate of the EPROM transistor. As long as the transistor has not been programmed into the OFF state, the word line can cause the "bit line", which is connected to a logic block input to be pulled to logic zero. In both the Altera and Plus Logic FPGAs, many EPROM transistors, each driven by a different word line, are connected to the same bit line. Since a pull-up resistor is present on the bit line, this scheme allows the EPROM transistors to not only implement connections but also to realize wired-AND logic functions. A disadvantage of this approach is that the resistor consumes static power.

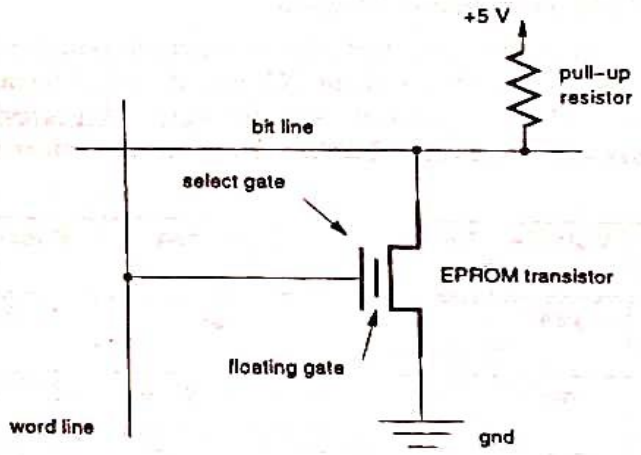


Figure 2.5 - *EPROM Programming Technology.*

One advantage of EPROM transistors is that they are re-programmable but do not require external storage. However, unlike static RAM, EPROM transistors cannot be re-programmed in-circuit.

The EEPROM approach (which is used in the FPGAs offered by Advanced Micro Devices) is similar to the EPROM technology except that EEPROM transistors can be re-programmed in-circuit. The disadvantage of using EEPROM transistors is that they consume about twice the chip area as

EPROM transistors and they require multiple voltage sources (for reprogramming) which might not otherwise be required.

❖ **Discuss summary of Programming Technologies**

2.1.4 Summary of Programming Technologies

Table 2.1 lists some of the characteristics of the various programming technologies that have been discussed in this section. The second column from the left gives an indication of whether or not the programming element is volatile, and the third column states if the element is re-programmable. The amount of area consumed by the programming elements is indicated, in relative terms, in the fourth column. While each company that produces FPGAs may use a different base CMOS process, the numbers shown in the table are all normalized to 1.2 μm CMOS. The fifth and six columns list the series resistance of the programming element (when in the ON state) and the capacitance that the element adds to each wire to which it is attached, respectively. The resistance and capacitance numbers are approximate and are meant to provide a relative measure for the various elements.

Programming Technology	Volatile	Re-Prog	Chip Area	R (ohm)	C (ff)
Static RAM Cells	yes	in circuit	large	1 - 2 K	10 - 20 ff
PLICE Anti-fuse	no	no	small anti-fuse, large prog. trans.	300 - 500	3 - 5 ff
ViaLink Anti-fuse	no	no	small anti-fuse, large prog. trans.	50 - 80	1.3 ff
EPROM	no	out of circuit	small	2 - 4 k	10 - 20 ff
EEPROM	no	in circuit	2x EPROM	2 - 4 k	10 - 20 ff

Table 2.1 - *Characteristics of Programming Technologies.*